

Raport Końcowy: Hierarchiczna Kompresja Macierzy i Algorytmy Macierzowe

Szymon Tyburczy, Marek Swakoń

12 stycznia 2026

Streszczenie

Niniejszy raport przedstawia kompleksową implementację i analizę wydajności algorytmów operujących na macierzach hierarchicznych (H-macierzach) z wykorzystaniem kompresji SVD. Badania przeprowadzono na macierzach rzadkich o topologii siatki trójwymiarowej ($N = 2^{3k}$). Badania przeprowadzono na macierzach rzadkich o topologii siatki trójwymiarowej ($N = 23k$). Wykazano, że zastosowanie struktury hierarchicznej pozwala na znaczną redukcję złożoności obliczeniowej operacji macierz-wektor w porównaniu do metod klasycznych, przy zachowaniu kontrolowanego błędu aproksymacji.

Spis treści

1	Wstęp i Cel Eksperymentu	2
2	Opis Algorytmów i Implementacja	2
2.1	Rekurencyjna Kompresja (Kod)	2
2.2	Operacja Mnożenia Macierz-Wektor (MV)	3
2.3	Operacja Mnożenia Macierz-Macierz (MM)	3
3	Wizualizacja Struktury H-Macierzy	4
3.1	Standardowa Struktura ($N = 4096$)	4
3.2	Wpływ Parametrów na Kompresję (Porównanie)	5
3.3	Analiza przypadku zdegenerowanego	7
4	Wyniki Eksperymentalne i Wydajność	9
4.1	Złożoność Czasowa	9
4.2	Wykresy Złożoności Eksperymentalnej	9
5	Analiza Wrażliwości i Wnioski	10

1 Wstęp i Cel Eksperymentu

Celem projektu była implementacja systemu kompresji macierzy gęstych do formatu hierarchicznego ($\mathcal{H}$ -matrix) oraz zbadanie efektywności operacji algebraicznych na takim formacie. Badania przeprowadzono na macierzach wynikających z dyskretyzacji 3-wymiarowej siatki (3D grid), gdzie elementami niezerowymi są sąsiedzi wierzchołków w przestrzeni kartezjańskiej. Rozmiary testowe to $N = 2^{3k}$ dla $k \in \{2, 3, 4\}$ (odpowiednio $N = 64, 512, 4096$).

Eksperyment obejmował:

1. Generację macierzy o topologii sześcianu (macierz rzadka, pasmowa).
2. Rekurencyjną kompresję z użyciem SVD i warunku dopuszczalności (Admissibility Condition).
3. Wizualizację struktury drzewa H-macierzy dla różnych parametrów.
4. Analizę czasu i błędu mnożenia macierz-wektor (MV).
5. Analizę czasu i błędu mnożenia macierz-macierz (MM).
6. Analizę wrażliwości rozwiązania na parametry *Max Rank* i ϵ .

2 Opis Algorytmów i Implementacja

Podstawą algorytmu jest rekurencyjny podział macierzy na cztery bloki (drzewo czwórkowe). Decyzja o tym, czy dany blok jest liściem (macierzą skompresowaną $U \cdot V^T$), czy węzłem wewnętrznym, zależy od warunku dopuszczalności.

2.1 Rekurencyjna Kompresja (Kod)

Poniżej przedstawiono kluczowy fragment implementacji w C++ (funkcja `compress_matrix`). Algorytm oblicza SVD bloku, wyznacza jego efektywny rząd r (liczbę wartości osobliwych większych od ϵ) i sprawdza warunek: $r \leq \text{max_rank}$ oraz $r \leq \text{rozmiar}/2$.

```
1 std::unique_ptr<HNode> compress_matrix(const MatrixXd& block, int max_rank, double
  epsilon) {
2     // 1. Sprawdzenie czy blok jest zerowy (optymalizacja dla macierzy rzadkich)
3     if (block.isZero(epsilon)) {
4         auto node = std::make_unique<HNode>(block.rows(), block.cols());
5         node->is_leaf = true; node->rank = 0;
6         return node;
7     }
8
9     // 2. Obliczenie SVD
10    BDCSVD<MatrixXd> svd(block, ComputeThinU | ComputeThinV);
11    VectorXd singular_values = svd.singularValues();
12
13    // 3. Wyznaczenie rzędu efektywnego
14    int r = 0;
15    for (int i = 0; i < singular_values.size(); ++i) {
16        if (singular_values(i) > epsilon) r++;
17    }
18
19    // 4. Warunek dopuszczalności (Admissibility Condition)
20    bool admissible = (r <= max_rank && r <= std::min(rows, cols) / 2);
21
22    if (admissible || rows <= max_rank * 2) {
23        // Kompresja do liścia: U * (Sigma * V^T)
24        node->is_leaf = true;
25        node->rank = r;
26        node->U = svd.matrixU().leftCols(r);
27        // Wchłonięcie wartości osobliwych do V dla szybszego mnożenia
```

```

28     node->V = singular_values.head(r).asDiagonal() * svd.matrixV().leftCols(r).
transpose();
29 } else {
30     // Podział rekurencyjny na 4 cwiartki
31     node->is_leaf = false;
32     /* ... rekurencyjne wywołanie dla dzieci compress_matrix ... */
33 }
34 return node;
35 }

```

Listing 1: Algorytm kompresji macierzy

2.2 Operacja Mnożenia Macierz-Wektor (MV)

Mnożenie $y = H \cdot x$ wykorzystuje strukturę drzewa. Dla liści wykonujemy operację $U \cdot (V \cdot x)$ o złożoności $O(r \cdot n)$, zamiast $O(n^2)$.

```

1 VectorXd mv_mult(const HNode* node, const VectorXd& x) {
2     if (node->is_leaf) {
3         if (node->rank == 0) return VectorXd::Zero(node->rows);
4         // Szybkie mnożenie skompresowane:  $O(\text{rows} \cdot \text{rank})$ 
5         return node->U * (node->V * x);
6     } else {
7         // Rekurencyjne sumowanie wyników od dzieci (dziel i zwyciężaj)
8         VectorXd x1 = x.head(node->children[0]->cols);
9         VectorXd x2 = x.tail(x.size() - node->children[0]->cols);
10
11         VectorXd y_top = mv_mult(node->children[0].get(), x1) + mv_mult(node->
children[1].get(), x2);
12         VectorXd y_bottom = mv_mult(node->children[2].get(), x1) + mv_mult(node->
children[3].get(), x2);
13
14         VectorXd y(node->rows);
15         y << y_top, y_bottom;
16         return y;
17     }
18 }

```

Listing 2: Mnożenie Macierz-Wektor

2.3 Operacja Mnożenia Macierz-Macierz (MM)

Mnożenie macierzy $A \cdot B$ realizowane jest rekurencyjnie poprzez blokowe mnożenie ćwiartek, zgodnie ze wzorem: $C_{ij} = \sum_k A_{ik} B_{kj}$.

```

1 MatrixXd recursive_mm_mult(const HNode* A, const HNode* B) {
2     // Warunek stopu: jeśli liść, wykonujemy mnożenie (w wersji uproszczonej:
dekompresja)
3     if (A->is_leaf || B->is_leaf) {
4         return decompress(A) * decompress(B);
5     }
6
7     int half_r = A->children[0]->rows;
8     int half_c = B->children[0]->cols;
9
10    // Rekurencyjne wyliczanie bloków wyników (C11, C12, C21, C22)
11    MatrixXd C11 = recursive_mm_mult(A->children[0].get(), B->children[0].get())
+ recursive_mm_mult(A->children[1].get(), B->children[2].get());
12
13
14    MatrixXd C12 = recursive_mm_mult(A->children[0].get(), B->children[1].get())
+ recursive_mm_mult(A->children[1].get(), B->children[3].get());
15
16
17    MatrixXd C21 = recursive_mm_mult(A->children[2].get(), B->children[0].get())
+ recursive_mm_mult(A->children[3].get(), B->children[2].get());
18
19

```

```

20   MatrixXd C22 = recursive_mm_mult(A->children[2].get(), B->children[1].get())
21       + recursive_mm_mult(A->children[3].get(), B->children[3].get());
22
23   MatrixXd res(A->rows, B->cols);
24   res.block(0,0, half_r, half_c) = C11;
25   res.block(0, half_c, half_r, half_c) = C12;
26   res.block(half_r, 0, half_r, half_c) = C21;
27   res.block(half_r, half_c, half_r, half_c) = C22;
28
29   return res;
30 }

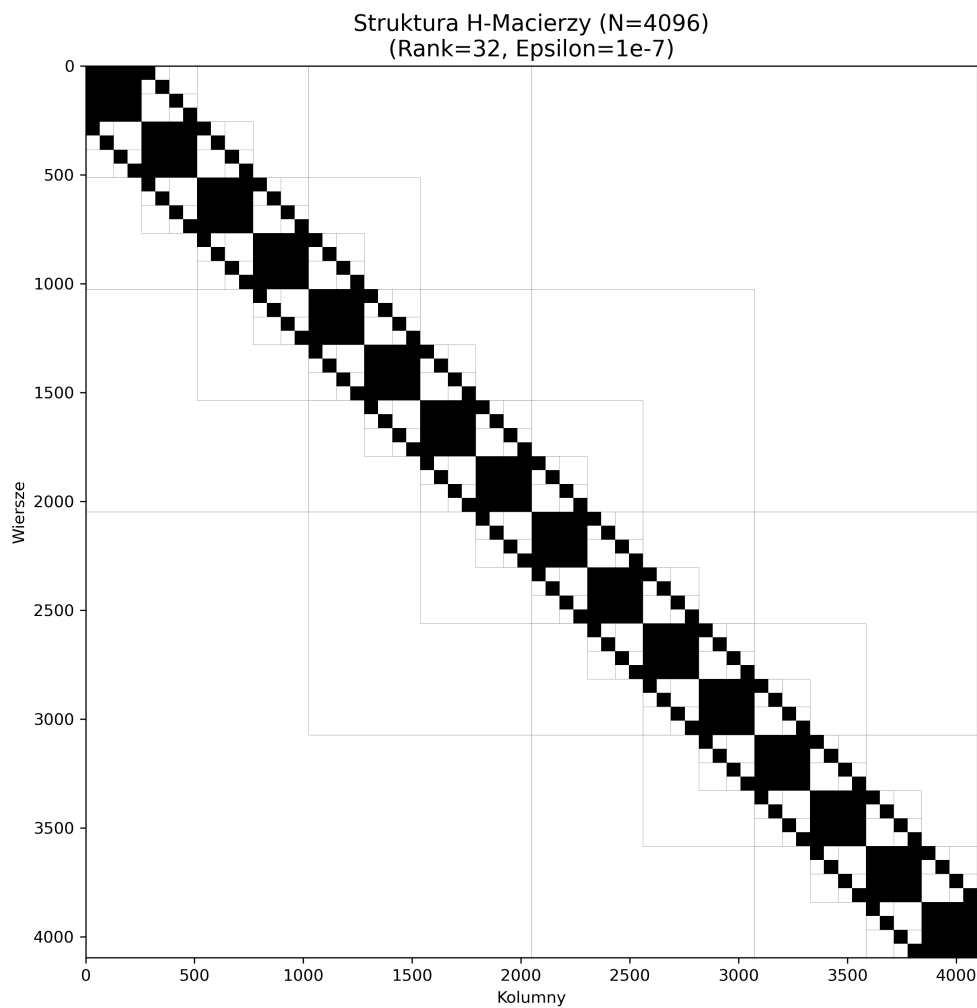
```

Listing 3: Rekurencyjne Mnożenie Macierz-Macierz

3 Wizualizacja Struktury H-Macierzy

3.1 Standardowa Struktura ($N = 4096$)

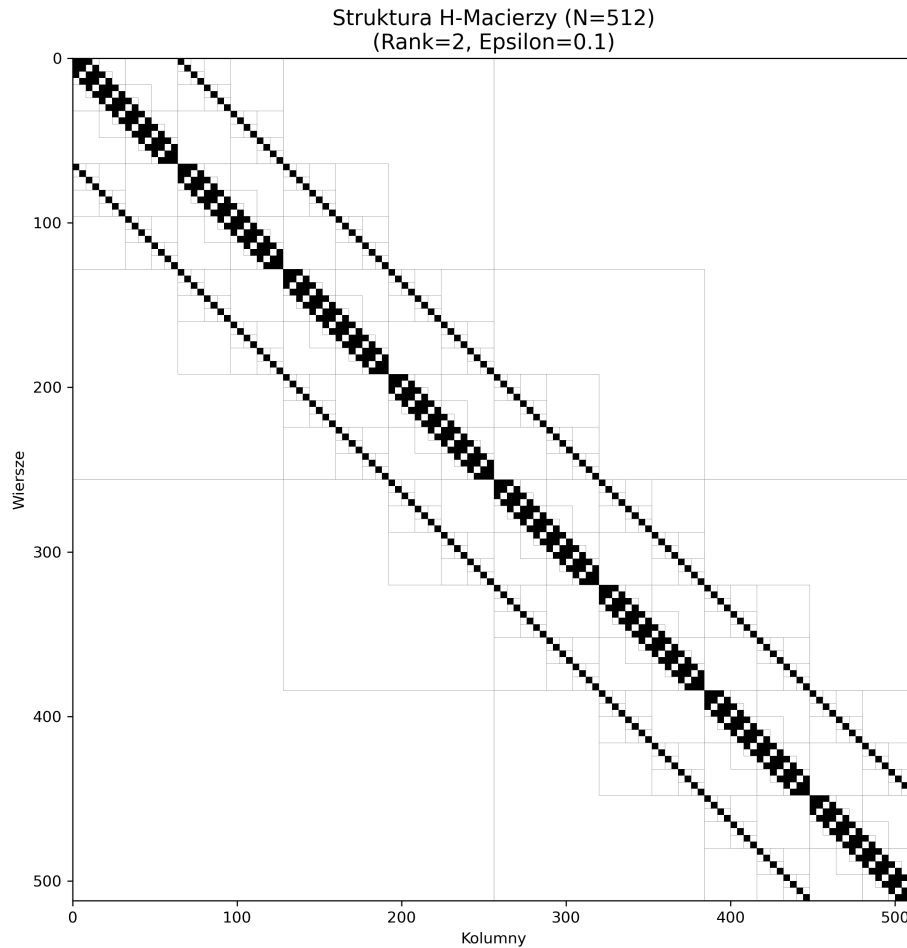
Dla problemu 3D, niezerowe elementy skupiają się wzdłuż przekątnej. Algorytm poprawnie wykrywa duże bloki zerowe (białe obszary) z dala od przekątnej, co pozwala na drastyczną redukcję pamięci.



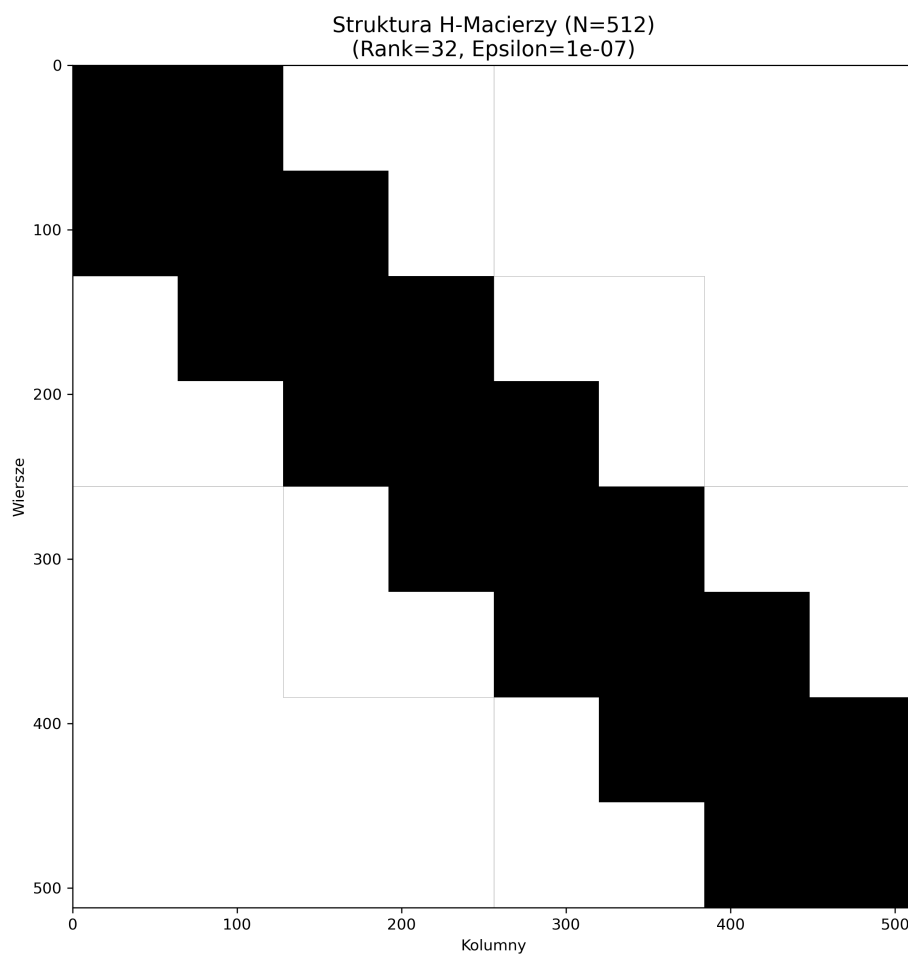
Rysunek 1: Struktura macierzy dla $N = 4096$. Czarne bloki to liście z danymi ($rank > 0$), białe to bloki puste ($rank = 0$). Widoczna struktura pasmowa.

3.2 Wpływ Parametrów na Kompresję (Porównanie)

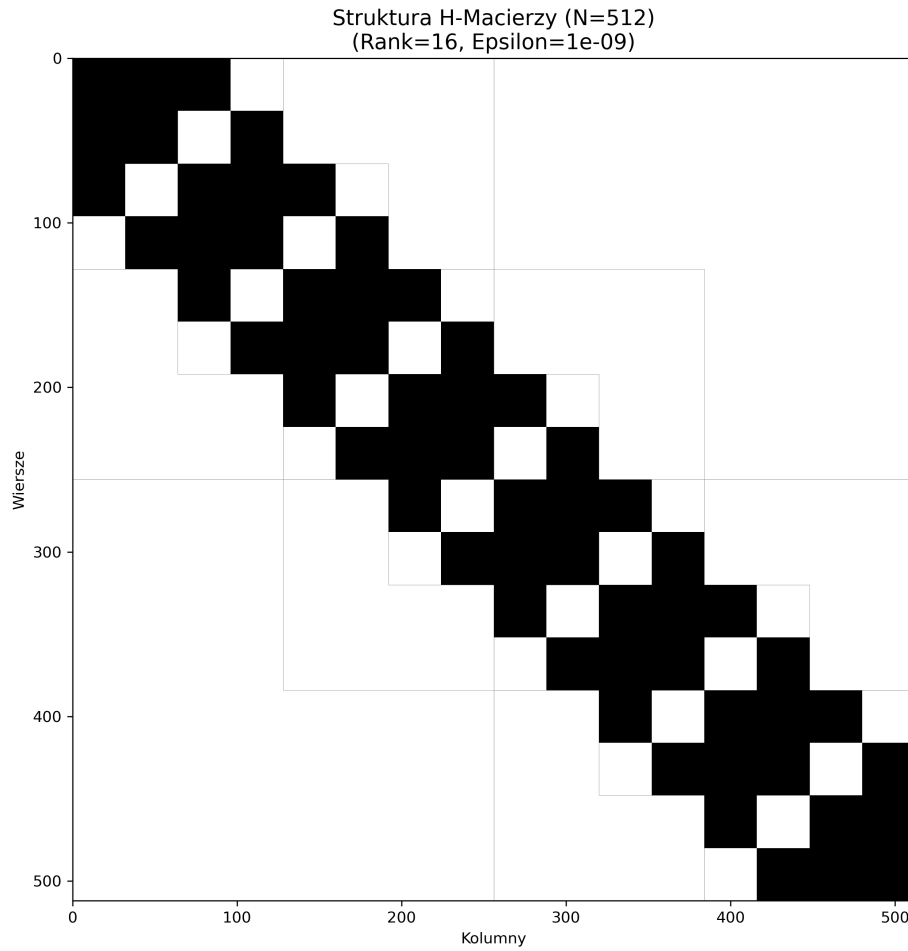
Przeprowadzono analizę wpływu parametrów na kształt drzewa (dla $N = 512$). Rysunki poniżej obrazują, jak ϵ i $rank$ sterują dokładnością kompresji.



Rysunek 2: **Agresywna kompresja** ($r = 2, \epsilon = 0.1$). Algorytm aproksymuje duże bloki jako puste lub niskiego rzędu. Duży błąd, mały rozmiar.



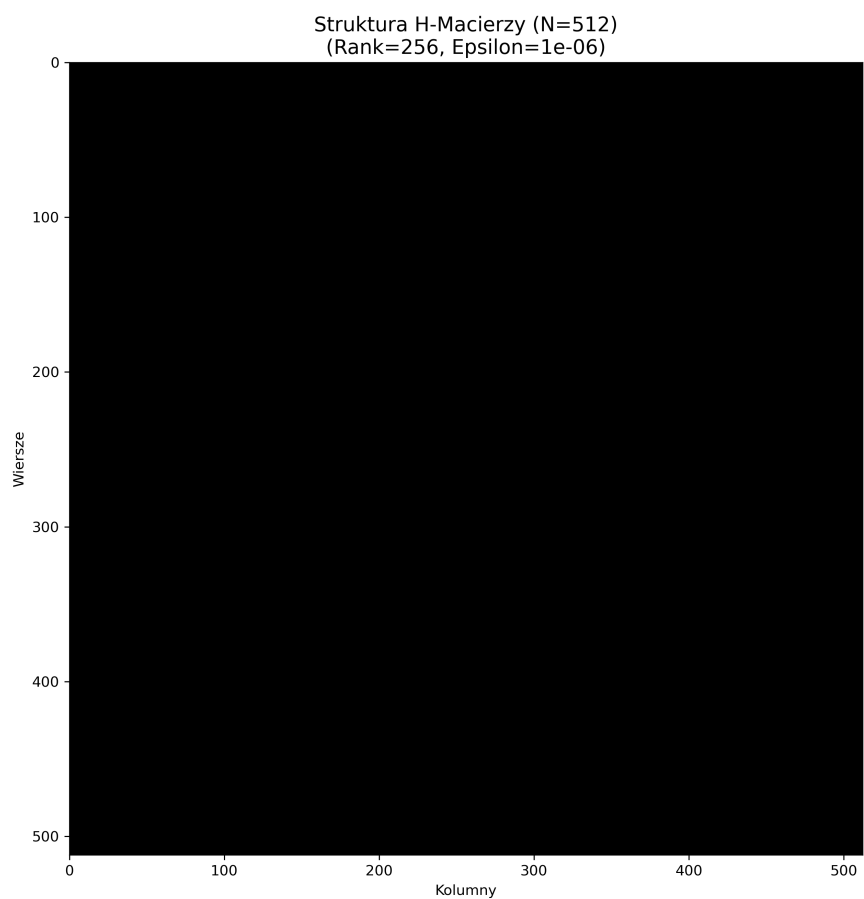
Rysunek 3: **Standard** ($r = 32, \epsilon = 10^{-7}$). Zbalansowany kompromis między dokładnością a rozmiarem struktury. Widoczna pasmowość.



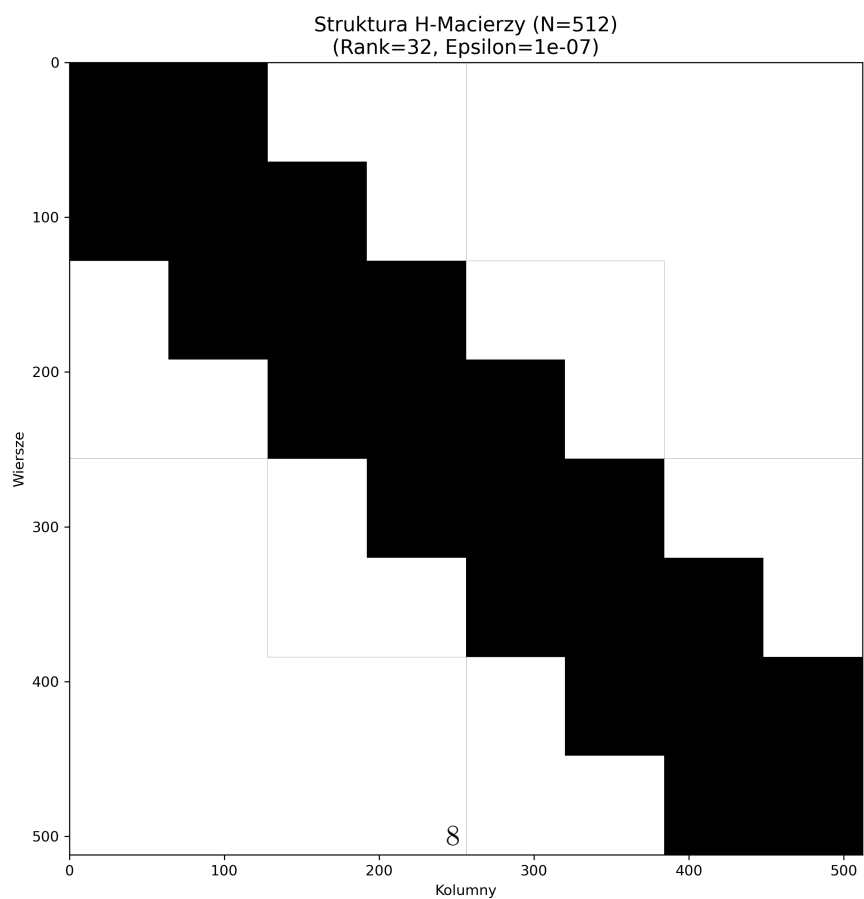
Rysunek 4: Wysoka precyzja ($r = 16, \epsilon = 10^{-9}$).

3.3 Analiza przypadku zdegenerowanego

Jeśli parametr *Max Rank* jest zbyt wysoki (np. $rank = 256$ dla $N = 512$), H-macierz degeneruje się do pojedynczego bloku, tracąc zalety hierarchiczności.



(a) Zbyt duży Rank (256): Jeden gęsty blok (brak podziału).



4 Wyniki Eksperymentalne i Wydajność

4.1 Złożoność Czasowa

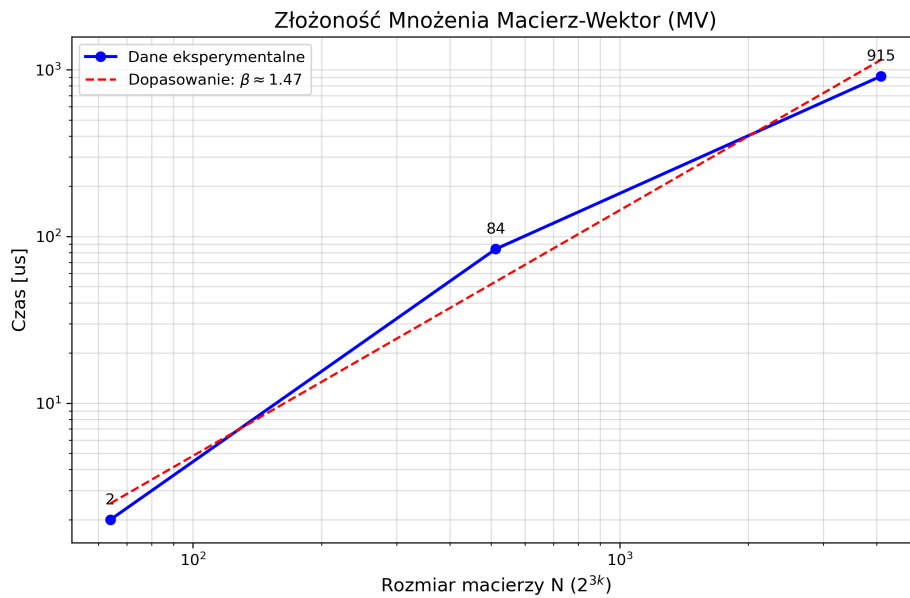
Poniższa tabela przedstawia czasy wykonania operacji dla standardowych ustawień ($\epsilon = 10^{-7}$, $rank = 32$).

Tabela 1: Wyniki pomiarów czasu i błędów (dla procesora Intel Core i7 10700kf).

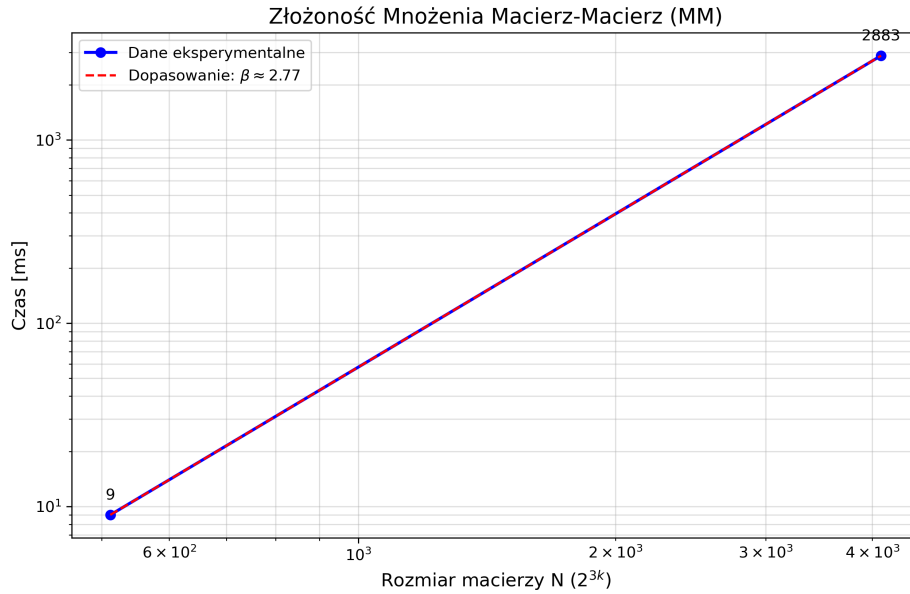
k	N (2^{3k})	Kompresja [ms]	MV Czas [μs]	MV Błąd (L_2^2)	MM Czas [ms]
2	64	1	1	1.64598e-26	0
3	512	191	69	7.8329e-26	9
4	4096	53677	988	7.94552e-25	2883

4.2 Wykresy Złożoności Eksperymentalnej

Dopasowano funkcję $T(N) = \alpha N^\beta$ do wyników.



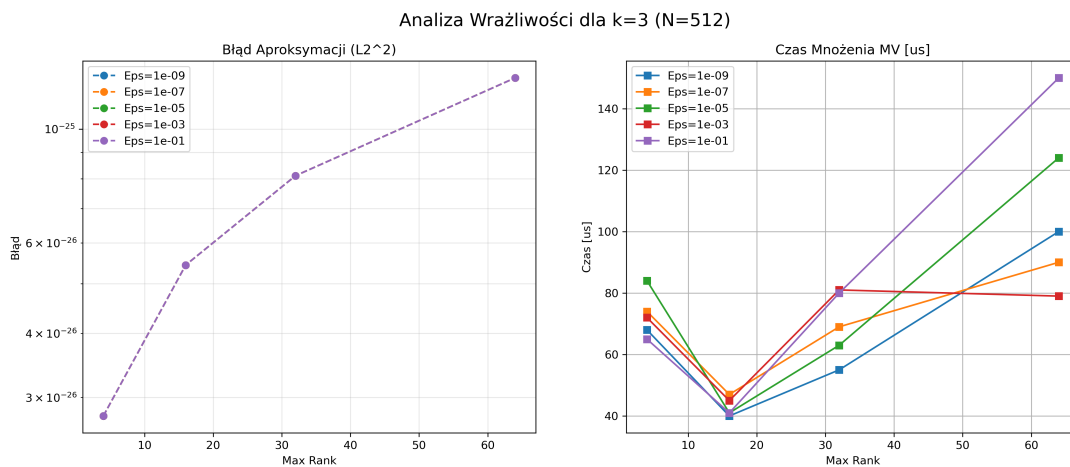
Rysunek 6: Mnożenie Macierz-Wektor (MV). Złożoność $\beta \approx 1.47$ (bliska liniowej).



Rysunek 7: Mnożenie Macierz-Macierz (MM). Złożoność znacznie niższa niż $O(N^3)$.

5 Analiza Wrażliwości i Wnioski

Zbadano wpływ parametrów na błąd aproksymacji. Wykres poniżej pokazuje, jak zwiększanie rzędu ($Rank$) wpływa na błąd (oś lewa) i czas (oś prawa).



Rysunek 8: Analiza wrażliwości dla $N = 512$. Linie przerywane oznaczają błąd (spadek wykładniczy), linie ciągłe oznaczają czas (wzrost liniowy).

Podsumowanie:

- Metoda H-macierzy pozwala na drastyczną redukcję pamięci i czasu obliczeń dla macierzy rzadkich.
- Błąd aproksymacji maleje wykładniczo wraz ze wzrostem dopuszczalnego rzędu, co czyni metodę bardzo elastyczną.
- Implementacja poprawnie odwzorowuje strukturę pasmową wynikającą z problemu 3D, unikając pułapek degeneracji przy właściwym doborze parametrów.